AI RED TEAM AGENT SIMULATOR

Final Demo Package

Exact 5-minute read-off script • terminal commands • CapCut cues

Target length	4:45–5:05
Demo target	travel_agent → qwen2.5:0.5b via Ollama
Assessment	deterministic / standard profile
Output	1920×1080, 16:9

Use this document as the teleprompter. Everything in quotation-style boxes is written to be read verbatim. Commands are in dark code blocks.

0:00–0:07 — Title Card

EDIT / VISUAL: cards/01_title.png

SAY This is my AI Red Team Agent Simulator — a local-first security assessment platform for authorized AI agents and LLM applications.

0:07–0:55 — Intro: Hook, Context, Pitch, Tech Stack

EDIT / VISUAL: cards/02_intro_problem_pitch_stack.png

SAY AI agents are increasingly given hidden instructions, access to tools, and sensitive context. The problem is that testing those boundaries is often manual, inconsistent, and difficult to reproduce.

I built this project to make that workflow repeatable: discover an authorized AI target, validate its scope, run bounded security probes, evaluate the responses, and preserve the results as evidence-backed findings.

The core platform is written in Python. Typer and Pydantic power the typed command-line interface and validation layer, and Ollama lets me run the target models locally without sending assessment data to an external model API.

0:55–1:05 — Core Flow Poster

EDIT / VISUAL: cards/03_core_flow.png

SAY The core flow is inventory, target resolution, planning, probe execution, deterministic evaluation, and reporting. Now I'll show that workflow against one local agent.

1:05–3:08 — Core Flow: Live Walkthrough

EDIT / VISUAL: Terminal full-screen. Ghostty 20–22 pt. Crop out unrelated windows.

1:05–1:18 — Confirm the local model runtime

```
ollama list
```

SAY Everything in this demo runs locally. The target I'm using is backed by Qwen 2.5 through Ollama.

1:18–1:32 — Show the target inventory

```
redteam agents list
```

SAY The simulator keeps an inventory of enrolled and discovered AI agents. I'm going to assess travel_agent, which is a local Python target using qwen2.5:0.5b.

1:32–1:58 — Preview the assessment

```
redteam assess plan python://travel_agent --profile standard
```

SAY Before executing anything, I can preview the exact assessment plan. The tool revalidates the target and scope, checks the available capabilities, and shows the registered probes that would run.

This standard deterministic assessment includes bounded tests for prompt disclosure, prompt injection, synthetic-secret leakage, unsafe tool claims, output behavior, error leakage, and model metadata.

EDIT / VISUAL: Do not linger on the full plan. Zoom/crop to authorization_required, probe categories, and deterministic: yes.

1:58–2:28 — Run the assessment

```
redteam assess run python://travel_agent \
  --authorization "I own this local synthetic target and authorize bounded testing."
```

SAY Active testing requires an explicit human authorization statement. The simulator now sends only the registered bounded probes to this exact target, stores the responses as evidence, and evaluates them using deterministic security rules.

SAY The run completed with no execution errors, full coverage, and one security finding.

2:28–2:40 — Capture the latest run ID

```
RUN_ID=$(basename "$(ls -dt reports/runs/run_* | head -1)")
echo "$RUN_ID"
```

SAY Each assessment is stored as a separate run, so I can inspect the findings and coverage independently.

2:40–2:55 — Show the finding

```
redteam reports findings "$RUN_ID"
```

SAY The evaluator created a confirmed high-severity synthetic-secret finding because the response repeated a controlled secret canary that it was expected to protect.

2:55–3:08 — Show coverage

```
redteam reports coverage "$RUN_ID"
```

SAY The important part is that every planned in-scope probe completed. The other tested categories passed, while the synthetic-secret category produced the finding.

3:08–3:30 — Finding Poster / Aha Moment

EDIT / VISUAL: cards/04_finding.png

SAY This is a controlled lab result, not a real credential leak. The probe uses a synthetic canary — a fake secret created specifically for testing. Because the evaluator knows the exact canary value, it can deterministically detect whether the target repeated protected data and create an evidence-backed finding.

3:30–4:12 — Behind the Scenes: Architecture

EDIT / VISUAL: cards/05_architecture.png

SAY Behind the CLI, the architecture is split into typed layers. Typer handles the command surface, Pydantic validates structured objects such as targets, plans, probes, and findings, the target resolver applies scope and capability checks, and the assessment engine executes only registered operations.

Two parts were especially important to get right. First, the system fails closed: active probes require an exact in-scope target, an authorization statement, and bounded request and time budgets.

Second, evaluation is evidence-based. Responses are normalized into findings using deterministic rules, and the run artifacts include coverage data and SHA-256 integrity manifests so the saved evidence can be verified later.

4:12–4:38 — Graceful Handling / Edge Cases

EDIT / VISUAL: cards/06_graceful_handling.png

SAY I also designed incomplete or unsafe states to be explicit. Missing authorization blocks active testing. Unapproved public targets are denied by default. If an integration is unavailable, that step is marked unavailable or skipped instead of being counted as a security pass. And unexpected failures return concise user-facing errors, with sanitized diagnostics available in debug mode.

4:38–5:00 — Wrap-Up & Future Work

EDIT / VISUAL: cards/07_wrap_thank_you.png

SAY The key takeaway is that this project turns AI-agent security testing into one repeatable workflow: discover, authorize, probe, evaluate, and preserve evidence.

The project already supports additional target types and adaptive assessment workflows. The next improvements I would focus on are stronger benchmarking for adaptive attack strategies and richer replayable multi-turn evaluation.

That’s my AI Red Team Agent Simulator. Thanks for watching.

Full Command Block — Copy/Paste

```
source .venv/bin/activate

ollama list

redteam agents list

redteam assess plan python://travel_agent --profile standard

redteam assess run python://travel_agent \
  --authorization "I own this local synthetic target and authorize bounded testing."

RUN_ID=$(basename "$(ls -dt reports/runs/run_* | head -1)")
echo "$RUN_ID"

redteam reports findings "$RUN_ID"

redteam reports coverage "$RUN_ID"
```

CapCut / Recording Settings

- Record only the terminal window or a clean full-screen terminal; hide browser and desktop clutter.
- Set Ghostty to 20–22 pt before recording so text is readable at 1080p.
- Edit on a 1920×1080, 16:9 CapCut canvas.
- Export 1080p, 30 fps, H.264, 20–30 Mbps. If source is 60 fps, keep 60 fps throughout.
- Cards are already rendered at approximately 1920×1080 and should stay at 100% scale when possible.
- Use short hard cuts/dissolves, not flashy transitions.
- Music: copyright-safe dark synth / minimal electronic instrumental; 12–15% intro/outro, 4–7% under speech.